UNIVERSIDADE DE FORTALEZA
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO: CIÊNCIA DA COMPUTAÇÃO

ORDENAÇÃO SERIAL VERSUS PARALELA EM JAVA: UM ESTUDO DE DESEMPENHO COM FORK/JOIN

Aluno 1: Davi Uchoa - 2410494.
Aluno 2: Thiago Maranhão - 2410413.

Resumo

Este trabalho compara o desempenho de versões seriais e paralelas de quatro algoritmos de ordenação (Merge Sort, Quick Sort, Selection Sort e Bubble Sort) implementadas em Java com o framework Fork/Join. Cada algoritmo foi executado com 1, 2, 4 e 8 threads, sobre vetores de 100 a 100.000 inteiros, em três arranjos de entrada (aleatório, já ordenado e inverso), com cinco medições por configuração. A partir dos tempos médios, calculamos speedup e eficiência. Os resultados mostram que a paralelização compensa de verdade nos algoritmos de dividir e conquistar: Merge Sort e Quick Sort chegaram a cerca de 3,2 e 3,3 vezes mais rápidos com 8 threads em 100.000 elementos aleatórios, ainda que com eficiência próxima de 40%. Já Selection Sort e Bubble Sort reagiram mal ao aumento de threads, porque o custo de coordenar tarefas passou a pesar mais que o trabalho dividido. Concluímos que o ganho depende menos de "usar mais threads" e mais de como o problema se divide.

Introdução

Hoje é difícil encontrar um computador com um único núcleo. Mesmo assim, boa parte dos programas que escrevemos no início da graduação continua rodando em uma única linha de execução, deixando os outros núcleos parados. A pergunta que motivou este projeto foi simples: se pegarmos algoritmos de ordenação conhecidos e os adaptarmos para rodar em paralelo, quanto de fato ganhamos?

A resposta intuitiva seria "quanto mais threads, mais rápido", e foi justamente essa intuição que quisemos testar. Dividir o trabalho em partes tem um custo (criar tarefas, esperar que terminem, juntar resultados), e nem todo algoritmo se deixa dividir com a mesma facilidade. Algoritmos como o Merge Sort e o Quick Sort têm uma estrutura recursiva natural, em que cada metade do vetor pode ser tratada de forma independente. Já o Selection Sort e o Bubble Sort, de natureza quadrática e muito sequenciais, exigem soluções menos óbvias.

Usamos o framework Fork/Join, disponível na biblioteca padrão de Java desde a versão 7, que foi pensado exatamente para problemas desse tipo (LEA, 2000; ORACLE, 2026). O objetivo do trabalho é medir, para cada algoritmo, o tempo das versões serial e paralela e discutir quando o paralelismo ajuda, quando fica indiferente e quando atrapalha.

Metodologia

O projeto foi escrito em Java 11 ou superior, sem bibliotecas externas. Ele reúne as implementações dos algoritmos, um programa de benchmark que grava os resultados em arquivos CSV e uma janela com gráficos para visualizar os dados. Cada algoritmo tem uma versão serial e uma paralela, e é sobre essa comparação que o estudo se apoia.

Versões paralelas. No Merge Sort e no Quick Sort, a estratégia é a clássica: o vetor é dividido, cada metade vira uma tarefa Fork/Join e, abaixo de um limite de 2.048 elementos, a tarefa passa a ordenar de forma serial, para não gastar mais tempo criando tarefas do que ordenando. No Merge Sort, a intercalação das duas metades continua sendo feita por uma única thread. No Selection Sort, a parte paralelizável é a busca pelo menor elemento do trecho restante, que é dividida em fatias entre as threads (e só quando o trecho tem ao menos 4.096 elementos). No Bubble Sort não existe uma divisão evidente, porque cada passada depende da anterior. Por isso a versão paralela usa a variante par-ímpar (odd-even transposition sort, descrita em KNUTH, 1998), em que cada fase compara pares de posições que não se sobrepõem e, portanto, podem ser processados ao mesmo tempo. Um detalhe importante: essa variante não é uma simples cópia paralela do Bubble Sort serial, e voltaremos a isso na discussão.

Dados de entrada. Testamos vetores de inteiros com 100, 1.000, 10.000 e 100.000 elementos, em três arranjos: aleatório, já ordenado e em ordem inversa. A semente do gerador é fixa (42), de modo que todas as execuções partem exatamente dos mesmos dados.

Execução e medição. Cada configuração foi executada cinco vezes, sobre uma cópia nova do vetor original, e o tempo foi medido com System.nanoTime. As versões paralelas rodaram em pools de 1, 2, 4 e 8 threads (8 é o número de núcleos que a JVM reportou na máquina de teste). Ao todo, são 1.200 medições registradas.

Métricas. A partir da média dos cinco tempos, calculamos o speedup, que é o tempo serial dividido pelo tempo paralelo, e a eficiência, que é o speedup dividido pelo número de threads. Um speedup de 4 com 8 threads, por exemplo, significa eficiência de 50%. Também registramos desvio padrão, mínimo e máximo de cada configuração.

Resultados e Discussão

Nesta seção, olhamos primeiro o caso mais favorável ao paralelismo (entradas grandes e aleatórias) e depois os casos em que as coisas deram errado, que, a bem da verdade, foram os mais interessantes.

Entradas grandes: onde o paralelismo funciona

Com 100.000 números aleatórios, o Merge Sort passou de 6,87 ms (serial) para 2,18 ms com 8 threads, um speedup de 3,15. O Quick Sort foi um pouco além: de 5,65 ms para 1,70 ms, ou 3,33 vezes mais rápido. É um bom resultado, mas convém olhar a eficiência. Com 8 threads, ela ficou em 39% (Merge Sort) e 42% (Quick Sort), ou seja, cada thread contribuiu com menos da metade do que um paralelismo perfeito entregaria. Parte disso é esperada: no Merge Sort, a última intercalação junta o vetor inteiro e é feita por uma só thread, e no Quick Sort a primeira partição

também. Esse trecho sequencial limita o ganho total, como descreve a Lei de Amdahl (1967). Além disso, ordenar um vetor é uma tarefa que mexe muito na memória, e é plausível que o acesso à memória, e não o processador, seja o gargalo quando várias threads trabalham juntas. Não medimos isso diretamente, então tratamos como hipótese.

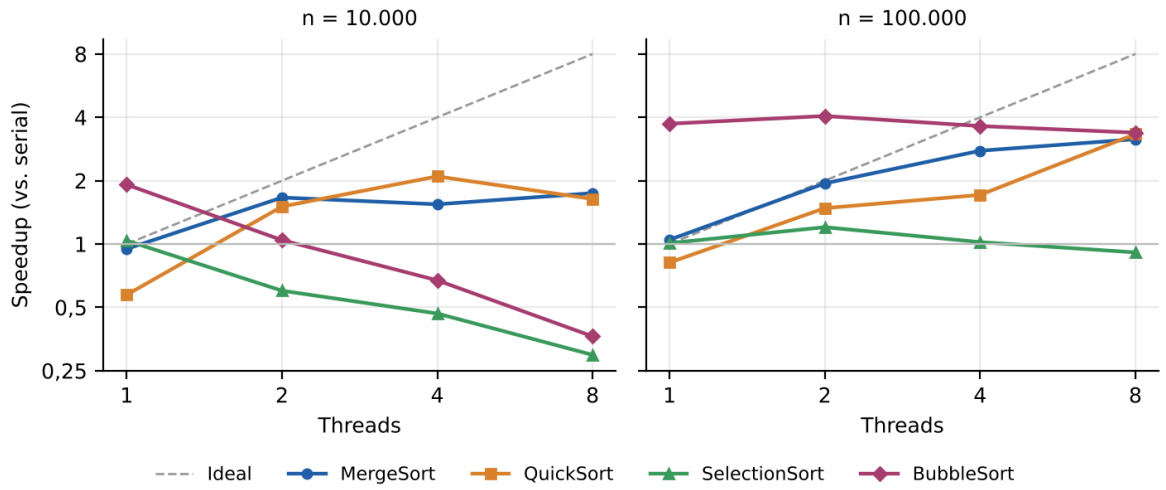


Figura 1 – Speedup em função do número de threads, com dados aleatórios (n = 10.000 à esquerda e n = 100.000 à direita). A linha tracejada indica o speedup ideal. Fonte: elaborada pelos autores (resumo.csv).

A Figura 1 resume a história inteira. Em 100.000 elementos (painel da direita), as curvas do Merge Sort e do Quick Sort sobem de forma consistente com o número de threads. Em 10.000 elementos (painel da esquerda), elas já sobem menos e começam a oscilar, e o Selection Sort e o Bubble Sort despencam abaixo de 1, isto é, ficam mais lentos que a versão serial.

Tabela 1 – Tempo médio (ms) com 100.000 elementos aleatórios. Fonte: elaborada pelos autores (resumo.csv). *O Bubble Sort paralelo usa a variante par-ímpar.

Algoritmo	Serial	1 thread	2 threads	4 threads	8 threads	Speedup (8)
Merge Sort	6,87	6,55	3,54	2,48	2,18	3,15
Quick Sort	5,65	6,92	3,82	3,30	1,70	3,33
Selection Sort	1.602,39	1.586,78	1.332,62	1.569,05	1.756,01	0,91
Bubble Sort*	7.666,03	2.055,32	1.888,67	2.108,36	2.266,38	3,38

Entradas pequenas: o custo de coordenar

Com poucos elementos, paralelizar não vale a pena. No Merge Sort, o speedup com 8 threads foi de 0,74 para n = 100, 1,05 para n = 1.000 e só então subiu para 1,75 e 3,15 (Figura 2). Com mil elementos, a tarefa inteira dura algo como 50 microssegundos, e criar e sincronizar tarefas custa uma fração relevante disso. Um aviso sobre o Quick Sort: o speedup de 1,31 em n = 100 não deve ser lido como vitória do paralelismo. Nessa escala, os tempos são tão curtos (da ordem de centésimos de milissegundo) e o desvio padrão tão grande em relação à média que a diferença está dentro do ruído de medição.

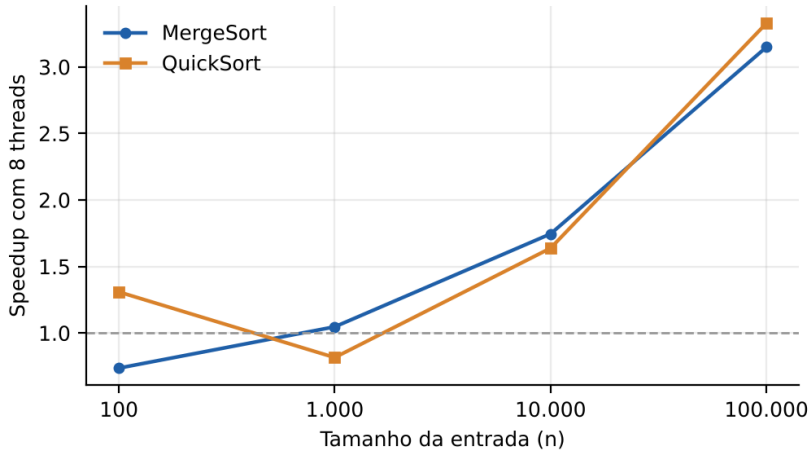


Figura 2 – Speedup do Merge Sort e do Quick Sort com 8 threads, por tamanho da entrada (dados aleatórios). Fonte: elaborada pelos autores (resumo.csv).

Selection Sort: mais threads, pior resultado

O Selection Sort foi o caso mais frustrante. Em 100.000 elementos, o melhor speedup foi de apenas 1,20 (com 2 threads), e com 8 threads o tempo ficou até pior que o serial (1.756 ms contra 1.602 ms, speedup de 0,91). Em 10.000 elementos, o efeito é bem mais dramático: o serial levou 17,5 ms e a versão com 8 threads levou 58,7 ms, quase o triplo. A explicação que nos parece mais razoável está na estrutura do algoritmo. A versão paralela precisa disparar uma rodada de tarefas Fork/Join a cada posição do vetor, ou seja, milhares de vezes, cada uma para uma busca de mínimo que dura poucos microssegundos. O custo de coordenação acaba maior que a própria busca, e quanto mais threads, mais coordenação. O tipo de entrada, por sua vez, quase não influenciou o Selection Sort, o que é coerente: ele faz a mesma varredura completa em qualquer arranjo.

Bubble Sort: um resultado que pede cautela

À primeira vista, o Bubble Sort paralelo parece o grande campeão, com speedups de 3,7 (aleatório, 100.000 elementos, 1 thread) e até 7,8 (entrada inversa). Só que há um detalhe que não podemos ignorar: esse ganho aparece já com uma única thread, o que mostra que ele não vem do paralelismo. Como mencionado na metodologia, a versão paralela é a variante par-ímpar, que é outro algoritmo. Nossa hipótese é que o padrão de acesso e de comparações dessa variante seja mais amigável ao processador e ao compilador JIT do que o laço do Bubble Sort clássico, mas não testamos isso. O que os dados mostram com clareza é o que acontece depois: acrescentar threads não melhora, e em alguns casos piora. Em 10.000 elementos aleatórios, a versão com 1 thread levou 20,8 ms, e a com 8 threads, 109,4 ms. O motivo é parecido com o do Selection Sort, já que cada fase exige uma sincronização completa. Há ainda o caso do vetor já ordenado, em que o Bubble Sort serial termina em 0,028 ms (ele percebe que não houve trocas e para na primeira passada) e a versão paralela precisa de ao menos duas fases completas, ficando mais lenta (0,41 de speedup). Ou seja, uma comparação justa entre os dois exigiria usar o mesmo algoritmo nas duas versões.

Efeito do tipo de entrada e limites do estudo

Entre os algoritmos eficientes, o tipo de entrada importou. O Quick Sort, com o pivô escolhido no meio do vetor, ordenou 100.000 elementos já ordenados em 0,95 ms, contra 5,65 ms nos aleatórios, e manteve speedups de cerca de 2,8 a 3,0 com 8 threads nesses casos. O Merge Sort, por fazer sempre o mesmo trabalho de intercalação, oscilou menos entre os arranjos (6,87 ms nos aleatórios e cerca de 2 ms nos outros dois).

Vale registrar o que limita estas conclusões. Foram apenas cinco medições por configuração, sem uma fase de aquecimento da JVM, de modo que as primeiras medições podem carregar custo de compilação JIT, e o desvio padrão é alto nos tamanhos pequenos. Todos os testes rodaram em uma única máquina, e como o speedup usa a média dos tempos, ele é sensível a medições fora do padrão. Por isso, preferimos nos apoiar nas tendências (entradas grandes, diferenças de várias vezes) e não em diferenças pequenas entre configurações.

Conclusão

O experimento mostrou que paralelizar não é um interruptor que se liga e pronto. Os algoritmos de dividir e conquistar, Merge Sort e Quick Sort, tiveram ganhos reais, de cerca de 3 vezes com 8 threads em 100.000 elementos, embora com eficiência de aproximadamente 40%, por causa das etapas sequenciais e, provavelmente, do acesso à memória. Em entradas pequenas, o paralelismo não compensou. Selection Sort e Bubble Sort, que dependem de muitas sincronizações curtas, ficaram mais lentos à medida que adicionamos threads.

Há ainda um aprendizado sobre método: o resultado aparentemente espetacular do Bubble Sort veio de uma troca de algoritmo, não de paralelismo, e só percebemos isso ao olhar a coluna de 1 thread. Como trabalhos futuros, sugerimos incluir aquecimento da JVM e mais repetições, comparar o Bubble Sort serial com a variante par-ímpar também em modo serial, testar outros limites de corte para as tarefas e repetir os testes em máquinas com números diferentes de núcleos.

Referências

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: AFIPS SPRING JOINT COMPUTER CONFERENCE, 1967, Atlantic City. **Proceedings** [...]. New York: ACM, 1967. p. 483-485.

CORMEN, T. H. et al. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

KNUTH, D. E. **The art of computer programming**: sorting and searching. 2. ed. Reading: Addison-Wesley, 1998. v. 3.

LEA, D. A Java Fork/Join framework. In: ACM JAVA GRANDE CONFERENCE, 2000, San Francisco. **Proceedings** [...]. New York: ACM, 2000. p. 36-43.

ORACLE. **Fork/Join**: the Java tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/essential/concurrency/forkjoin.html. Acesso em: 30 out. 2026.